

Raspberry Pi

SuperCapacitor UPS with Power Switch

User Manual & Technical Reference

Based on the Hackaday.io project by D. E. This document covers the complete feature set, technical specifications, operating modes, Raspberry Pi software setup, and known pitfalls of the SuperCapacitor UPS board.

Source	hackaday.io/project/175895
Author	D. E. (stb)
Published	November 2020
License	CC BY-NC-SA 4.0
Buy	lectronz.com/products/smart-ups-supply-for-raspberry-pi-power-button
Firmware	github.com/STB3/SuperCapPi

1 Overview

The Raspberry Pi SuperCapacitor UPS with Power Switch is a Pi HAT-compatible power supply board designed to solve common reliability problems in always-on Raspberry Pi deployments. Unlike battery-based UPS solutions, it uses supercapacitors for energy storage — eliminating fire risk, maintenance cost, and capacity degradation over time.

The board was originally designed for arcade machines but is equally suited to NAS boxes, home servers, and IoT nodes — any application where an unexpected power cut must not corrupt the SD card.

1.1 Feature List

- Stable 5 V power supply with short-time UPS function
- Bridges power interruptions <2 s completely unnoticed
- Robust USB-C power input (replaces fragile micro-USB)
- Graceful-shutdown triggered by physical power button
- No maintenance — supercapacitors do not age or explode
- Connector for external power button
- Connector for optional expansion board (relay + power LED) — loads switched via relay do not drain the supercaps
- State memory: board remembers its last state across power cycles

1.2 Optional Add-on Board

- External power LED indicator
 - Relay to switch high-current loads (up to 115 V / 230 V AC, 6 A)
-

2 How It Works

The design is centred on the **LTC4041** supercapacitor charger IC from Linear Technology (now Analog Devices), following its application note closely.

A dedicated **STM32L011 microcontroller** handles all logic: monitoring power-fail signals from the LTC4041, responding to button presses, communicating with the Raspberry Pi over GPIO, and persisting state in its internal EEPROM.

2.1 Microcontroller Functions

Short-glitch UPS bridging

The supercapacitor bank absorbs power interruptions shorter than 2 seconds without the Raspberry Pi ever seeing a voltage dip. No software action is needed.

Shutdown on power loss (>2 s)

If the LTC4041 signals a power-fail condition that lasts longer than 2 seconds, the microcontroller pulls the **Power Good** GPIO line low to notify the Pi. After approximately 25 seconds the supply rail to the Pi is cut entirely.

Shutdown on button press

Pressing the power button immediately pulls Power Good low. The Pi has 25 seconds to complete its shutdown before the supply is removed.

Shutdown via Linux shutdown command

The Raspberry Pi drives **GPIO 24** or **GPIO 26** high as part of its shutdown sequence. The microcontroller detects this and cuts the supply after 25 seconds — allowing a fully software-initiated, hardware-confirmed power-off.

State memory (EEPROM with wear levelling)

The current operating state is written to EEPROM on every transition. A wear-levelling algorithm guarantees more than **1 million write cycles**. After any power cycle the board returns to exactly the state it was in before.

3 Operating States

The microcontroller maintains the following named states. Transitions are always persisted to EEPROM before the μ C returns to sleep.

State name	Description
ON_powered	Normal operation — Pi is running, mains present
OFF_powered	Shutdown initiated by power-button press, mains present
APPshtdn_powered	Shutdown initiated by Linux shutdown command, mains present
APPshtdn_unpowered	Shutdown by OS, then mains removed — board stays off
ON_unpowered	Pi was running when mains was removed — will restart when mains returns
OFF_unpowered	Pi was off when mains was removed — stays off, needs button to start

3.1 Power-on Behaviour Summary

- Mains removed while ON → reconnecting mains automatically restarts the Pi
- Mains removed while OFF → reconnecting mains keeps the Pi off; press button to start
- Switched off by button → must press button again to restart
- Shutdown by OS command → press button or reconnect mains to restart

4 DC Specifications

For best performance use the **official Raspberry Pi USB-C power supply (3 A)**. See Section 7 for important caveats about other supplies.

Parameter	Value
Input voltage	5.0 V ... 5.2 V
Output voltage (mains present)	Same as input voltage
Output voltage (running on supercaps)	4.72 V ... 4.88 V
Output current (mains present)	Limited by wall plug rating
Supercapacitor charge current	1 A max.

Min. wall-plug current rating	3 A (Pi load + 1 A charge current)
Backup time — 500 mA load, 25 F	≈ 60 s (supply cut after 20 s)
Backup time — 1000 mA load, 25 F	≈ 25 s (supply cut after 20 s)

5 Measured Performance

5.1 Raspberry Pi 3B + Official TFT Display

- Operating current: approximately 1 A
- Shutdown duration after button press: approximately 4 s
- Steady-state current after shutdown: 300 mA
- Power supply cut: 20 s after shutdown completes
- Supercap voltage drop during shutdown (35 F caps): 600 mV (4.6 V → 4.0 V)

5.2 Raspberry Pi 4B 8 GB + Official TFT Display

- Slightly higher running current than the 3B
- Supercap voltage at end of shutdown cycle: ~3.7 V — still ample margin
- 35 F caps can be replaced with 25 F for a more compact fit (avoids physical clash with the LAN connector)

5.3 Notes on Capacitor Sizing

Based on measurements the designer concluded that 25 F supercapacitors provide sufficient backup time while being slightly shorter — avoiding mechanical interference with the Raspberry Pi's Ethernet connector. The shutdown timeout could also be reduced from 20 s to 15 s without loss of safety margin.

5.4 Relay Expansion Board Benefit

Connecting all ancillary 5 V loads through the optional relay expansion board significantly reduces current drawn from the supercapacitors during a backup event, extending runtime further.

6 Raspberry Pi Software Setup

6.1 Minimum Required Configuration

Only one change to `/boot/config.txt` is mandatory — enabling the GPIO shutdown overlay. This allows the board's power button to trigger a clean OS shutdown.

```
# /boot/config.txt
# Use default GPIO 3:
dtoverlay=gpio-shutdown
# OR use GPIO 22 (optional alternative):
dtoverlay=gpio-shutdown,gpio_pin=22
```

This configuration alone is sufficient if you only want to power the board on with the button or by plugging in the USB-C connector. **The steps below are only required if you also want the OS to signal the UPS board before shutting down.**

6.2 Optional: OS-Initiated Shutdown Signal

Two small C programs signal the UPS board at boot and shutdown via a dedicated GPIO line (GPIO 24 or GPIO 26). The required source files are available in the GitHub repository download section.

Install WiringPi

```
gpio -v # check whether already installed
```

If not installed, follow the instructions at wiringpi.com/download-and-install.

c-start — runs at boot

Sets the chosen GPIO as output and drives it LOW, signalling to the UPS that the OS is running.

```
gcc c-start_GPIO26.c -o c-start -lwiringPi
cp c-start /usr/local/bin/
```

c-stop — runs at shutdown/reboot

Sets the same GPIO HIGH, signalling to the UPS that a shutdown is in progress.

```
gcc c-stop_GPIO26.c -o c-stop -lwiringPi
cp c-stop /usr/local/bin/
```

6.3 systemd Service

The service **powercontrol-comms.service** wires the above binaries into the systemd startup and shutdown lifecycle.

```
# Copy service file:
sudo cp powercontrol-comms.service /etc/systemd/system/

# Verify it is present:
ls /etc/systemd/system/powercontrol-comms.service

# Enable and start:
sudo systemctl enable powercontrol-comms.service
sudo reboot
```

■ *Ensure c-start and c-stop are present in /usr/local/bin/ before rebooting.*

7 Known Pitfalls & Important Warnings

7.1 Switched Power Strips — Single-Pole Switching

Using a switched power strip that disconnects only one mains conductor can cause misbehaviour. The official Raspberry Pi USB-C supply can produce voltage pulses lasting several seconds via its internal Y-capacitors even after the strip switch is opened. These pulses can wake the UPS microcontroller and corrupt its state machine.

■ *Always remove the plug from the outlet, or use a power strip that disconnects both conductors simultaneously.*

7.2 USB-C Cables and Generic Supplies

- The input voltage must remain above the supercap minimum voltage (4.7 V) plus the LTC4041 charging headroom — aim for at least 5.0 V at the board input.
- Cheap 50 cm USB-C to USB-A cables tested showed up to 500 mV voltage drop under load.
- The official Raspberry Pi USB-C supply (3 A) is strongly recommended: low-resistance cable, 3 A rating, best voltage regulation.

7.3 Wall Plug Current Rating

The board imposes a built-in 3 A current limit on the combined load. The Pi's supply current and the supercap charge current (up to 1 A) are additive. If their sum exceeds 3 A the charge current is throttled — but Pi supply is not limited. The **minimum recommended wall plug rating is 3 A**.

8 Microcontroller Firmware

8.1 Hardware

The onboard microcontroller is an **STM32L011** (ultra-low-power ARM Cortex-M0+). Firmware source is published at github.com/STB3/SuperCapPi.

8.2 Architecture

- Fully interrupt-driven; the MCU sleeps between events
- Three interrupt sources: power button, LTC4041 power-fail/good output, GPIO from Raspberry Pi
- Each interrupt wakes the MCU, determines the next state, takes action (switch supply on/off), writes the new state to EEPROM, then returns to sleep
- Actions are only taken when mains power is present; without mains the new state (e.g. unpowered-off) is simply recorded

8.3 State Enumeration (from firmware source)

```
enum states {
    state_undefined    = 0,
    ON_powered,        // Unit is running
    OFF_powered,        // Shutdown via power button
    APPshdn_powered,    // Shutdown via OS command
    APPshdn_unpowered,  // Shutdown by OS, then mains removed
    ON_unpowered,       // Running when mains removed
    OFF_unpowered       // Was off when mains removed
};
```

9 Typical Use Cases

- Raspberry Pi NAS — protects ZFS/ext4 journals from corruption on power cut
- Home server / Home Assistant hub — eliminates manual shutdown before travel
- IoT gateway — recovers automatically after mains glitches
- Arcade cabinet — original design target; player-facing power button works natively
- Any embedded Pi application where SD-card corruption on power loss is unacceptable

10 Quick-Start Checklist

1. Mount the board on the Raspberry Pi GPIO header (HAT-compatible)
2. Connect the official Raspberry Pi USB-C power supply (3 A) to the board's USB-C input
3. Add `dtoverlay=gpio-shutdown` to `/boot/config.txt` and reboot
4. (Optional) Compile and install `c-start` / `c-stop`, then enable `powercontrol-comms.service`
5. (Optional) Connect the relay expansion board for high-current loads
6. (Optional) Wire an external power button to the expansion header

7. Test: press power button → Pi should initiate a clean shutdown and power off after ~25 s
8. Test: pull the USB-C plug while Pi is running → Pi should restart automatically when plugged back in

11 License

This project is released under the **Creative Commons Attribution-NonCommercial ShareAlike 4.0 International** (CC BY-NC-SA 4.0) licence.

- You may share and adapt the material for non-commercial purposes
- You must give appropriate credit and link to the licence
- Derivative works must be distributed under the same licence
- Full licence text: creativecommons.org/licenses/by-nc-sa/4.0/

*This manual was compiled from the Hackaday.io project page. All technical content is the work of the original author (D. E. / stb).
Manual layout CC BY-NC-SA 4.0.*